End-to-End MLOps Pipeline

Sentiment Analysis with DistilBERT — SST-2

Docker · GitHub Actions · Kaggle · Weights & Biases · Hugging Face

MLOps | PGD AI Program, IIT Jodhpur | Group 21

Draft — replace [INSERT] values with your actual W&B metrics, paste screenshots where marked, verify every link is public, then export to PDF.

1. Team & Contributions

Roll No.	Name	Contributions
G25AIT2079	**Rajath S M**	Lead — pipeline architecture, training script & Kaggle notebooks (v1/v2), W&B integration, Dockerfile, CI/CD workflows, repository admin.
G25AIT2069	Nilajit Sarkar	Data preparation & cleaning script, raw-data inspection and class-distribution analysis, W&B dashboard configuration, run testing.
G25AIT2130	Vishvesh Rajpurohit	Model selection & rationale, Hugging Face Hub push of the best model, inference-script support, model-card review.
G25AIT2142	Niketh Varma Tirumalaraju	GitHub Actions runs & repository secrets, Docker image build/test/push, report compilation, public-link verification.

All members contributed commits to the repository; contributions are visible in the GitHub commit history.

2. Live Links (all public)

Component	URL
GitHub Repository	github.com/Rajath-g25ait2079/mlops-sentiment-pipeline
Kaggle Notebook — V1	kaggle.com/rajathsmg25ait2079/mlops-sentiment-v1
Kaggle Notebook — V2	kaggle.com/rajathsmg25ait2079/mlops-sentiment-v2
Hugging Face Model	huggingface.co/Rajath-g25ait2079/distilbert-sst2-sentiment
Docker Image (Docker Hub)	hub.docker.com/r/rajathsmg25ait2079/mlops-a3-inference
W&B Project Dashboard	wandb.ai/g25ait2079/mlops-assignment3

Note: broken or private links score zero for that component — confirm each opens in an incognito window before submission.

3. Git Repository Setup (Task 1)

The repository is a public GitHub repo owned by Rajath S M (Admin). It ships with a README.md, .gitignore, and an MIT LICENSE. A develop branch holds active work and the main branch is protected: at least one pull-request approval is required before merging, so no member pushes directly to main. The other three members are added as Collaborators with Write access. This mirrors a real team workflow — feature work on develop, reviewed PRs into main.

[SCREENSHOT: Settings → Collaborators page showing all 4 members with roles, AND the

branch-protection rule on main]

4. Data Cleaning & Normalisation (Task 2)

We used the Stanford Sentiment Treebank (SST-2) binary-sentiment dataset from the Hugging Face Hub. The public test split has hidden labels, so we built our own splits from the labelled training split (~67k short movie-review sentences) to obtain real evaluation metrics.

Cleaning decisions and rationale: (1) Missing/empty values — rows with a null or blank sentence or label were dropped, since they carry no learnable signal. (2) Normalisation — text was lowercased and internal whitespace collapsed to single spaces; lowercasing matches the uncased WordPiece tokenizer of DistilBERT, avoiding redundant casing variants. (3) Duplicates — exact (text, label) duplicates were removed to prevent the same sentence leaking across train/val/test and inflating scores. (4) Length filter — empty-after-cleaning and pathologically long strings were filtered. (5) Class distribution — SST-2 is roughly balanced (~56% positive / 44% negative); we used a stratified split to preserve this ratio rather than risk skew.

Encoding & splits: Labels are integer-encoded and saved to id2label.json as {"0": "negative", "1": "positive"} — the only data artifact committed to git. The dataset was stratified-sampled to 20,000 rows (to fit Kaggle's free GPU budget) and split 80/10/10 into train/validation/test. Prepared CSVs are saved locally and git-ignored.

5. Model Selection Rationale (Task 3)

We selected distilbert-base-uncased from the Hugging Face Hub as the backbone of our sentiment classifier. Per its model card, DistilBERT is a distilled version of BERT-base that retains about 97% of BERT's language understanding while being roughly 40% smaller and 60% faster, with only ~67M parameters. This compact footprint suits Kaggle's free T4 GPU, letting both experiment versions train well within the weekly quota. The card reports a strong SST-2 GLUE score of 91.3, confirming the architecture fits binary sentiment classification — exactly our task. It is released under the permissive Apache-2.0 licence and is explicitly intended to be fine-tuned for sequence classification. Its uncased WordPiece tokenizer (30k vocab) aligns with our lowercasing step, and its very large fine-tune ecosystem makes troubleshooting straightforward.

Loading: we load the AutoTokenizer for the model and AutoModelForSequenceClassification with num_labels=2, passing id2label/label2id from our mapping file so predictions return human-readable labels.

6. Experiment Comparison — V1 vs V2 (Task 4)

Both versions fine-tune the same model on the same data and differ in exactly one hyperparameter — the learning rate — so the comparison is meaningful. All runs log training loss, validation loss, accuracy, weighted F1, and the full hyperparameter config to Weights & Biases.

Setting / Metric	Version 1	Version 2
Base model	distilbert-base-uncased	distilbert-base-uncased
Learning rate	3e-5	1e-5 (changed)
Epochs	3	3
Batch size	16	16

Setting / Metric	Version 1	Version 2
Training loss	[INSERT]	[INSERT]
Validation loss	[INSERT]	[INSERT]
Accuracy	[INSERT]	[INSERT]
Weighted F1	[INSERT]	[INSERT]

Which performed better and why: Based on the W&B comparison, [INSERT Version X] achieved the higher validation F1. *Expected interpretation (confirm against your dashboard): with epochs fixed at 3, the higher learning rate (3e-5) typically converges faster and reaches a better optimum within the budget, whereas 1e-5 tends to under-fit in the same number of steps; if 1e-5 instead generalises better, it is usually because the larger rate slightly overshoots on this small, clean dataset.*

[SCREENSHOT: W&B dashboard showing BOTH runs (run-v1 and run-v2) — Runs Comparison table with Accuracy, F1 and Loss]

7. Model on Hugging Face Hub (Task 5)

The best run's weights and tokenizer are pushed to the public repository Rajath-g25ait2079/distilbert-sst2-sentiment. The model URL is written to the W&B run summary (wandb.run.summary['huggingface_model']) so the experiment record links directly to the deployed artifact. The GitHub Actions inference workflow pulls the model from this URL.

8. Docker Design (Task 6)

Dockerfile choices: we start from python:3.11-slim to keep the image small and install only inference dependencies (torch, transformers, numpy, huggingface_hub) — training packages such as datasets and wandb are deliberately excluded. CPU-only PyTorch is installed from the official CPU wheel index to avoid pulling multi-gigabyte CUDA libraries. The Hugging Face model name is a build ARG HF_MODEL_NAME with a sensible default, so the image can be rebuilt for any checkpoint. requirements.txt is copied before the source to exploit Docker layer caching, the container runs as a non-root user, and the ENTRYPOINT runs the inference script, reading the text from the INPUT_TEXT env var or a CLI argument. The built image is published publicly to Docker Hub at rajathsmg25ait2079/mlops-a3-inference.

```
docker build --build-arg HF_MODEL_NAME=Rajath-g25ait2079/distilbert-sst2-sentiment -t rajathsmg25ait2079/mlops-a3-inference:latest .
docker run --rm -e INPUT_TEXT="This movie was a masterpiece." rajathsmg25ait2079/mlops-a3-inference:latest
docker push rajathsmg25ait2079/mlops-a3-inference:latest
```

Successful inference workflow log (paste from GitHub Actions):

[ACTIONS LOG: paste the log from a successful 'Inference' workflow run — the step that prints the predicted label and score]

9. GitHub Actions (Task 7)

Two workflows live in .github/workflows/. ci.yml runs on every push to develop (and PRs to main): it installs flake8 and lints / byte-compiles src/. inference.yml is triggered manually (workflow_dispatch) with a text input; it installs CPU PyTorch, pulls the published model and

classifies the text, reading HF_TOKEN from repository secrets. Training is never run in Actions — only on Kaggle GPUs. No tokens are committed; HF_TOKEN is stored under Settings → Secrets and variables → Actions.

[SCREENSHOT: Actions tab showing a successful CI run AND a successful manual Inference run]

10. Challenges & Learnings

What was hard. Keeping the Docker image small was the main challenge — installing PyTorch from the default index pulls heavy CUDA wheels, so we switched to the CPU-only wheel index, which cut image size and CI time substantially. Branch protection initially blocked direct pushes to main, which forced a proper pull-request workflow. Managing secrets across three platforms (Kaggle Secrets, GitHub Secrets, the HF write token) without hardcoding anything required care.

What we learned. Changing a single hyperparameter at a time is what makes a W&B comparison interpretable. Treating the model as a black box and investing effort in the surrounding pipeline — reproducible data prep, containerised inference, automated CI — is what makes the project production-style.

What we would do differently. Run a small hyperparameter sweep instead of two manual runs, add unit tests for the data-prep and inference functions to the CI workflow, and pre-bake the model into the Docker image so inference needs no network at runtime.